

Linux Kernel Hash Function 數學基礎與資訊安全議題

jhash2 / HalfSipHash / SipHash Trade-off Analysis

Experiment 1: Kernel-side microbenchmark

Linux kernel

OVS flow table

perf stat

microbenchmark

專題目標

分析 Linux kernel 中 `jhash2`、`hsiphash` 與 `siphash` 在效能與安全性上的取捨，並以 OVS flow table 作為後續實驗平台。

本次檢核點實作內容為 Experiment 1：

- 建立 kernel-side hash function microbenchmark
- 量測 cycles/hash 與 perf counters
- 先隔離 hash function 本身成本，再進入 OVS datapath

這樣可以避免一開始直接改 OVS 時，把 flow lookup、RCU、softirq 與 cache locality 等因素混在一起解讀。

為什麼比較這三種 hash ？

Linux kernel 會依照使用情境選擇不同 hash function 。

Hash	特性	定位
<code>jhash2()</code>	non-cryptographic hash，設計上偏向低延遲與高吞吐量 情境	OVS flow table baseline
<code>hsiphash()</code>	keyed 32-bit hash	security-performance middle ground
<code>siphash()</code>	keyed 64-bit hash 具備 PRF 特性，可抗 HashDoS 攻擊	stronger keyed hash baseline

問題：Linux 為什麼沒有全面用 SipHash 取代既有 hash ？

Experiment 1: Microbenchmark 設計

採用 out-of-tree kernel module 實作。

透過模組載入動態切換參數以執行指定的 hash function。

```
sudo insmod ./hash_microbench.ko \  
hash_type=0 \  
input_len=64 \  
iterations=10000000
```

Parameter	Meaning
hash_type=0,1,2	jhash2(), hsiphash(), siphash()
input_len	input buffer length
iterations	hash loop iterations

選擇 kernel module 的原因是可以直接呼叫 Linux kernel 內部 hash function
避免 user-space 測試後的移植跟實際核心環境存在差異。

實驗設計：input length

三種 hash function 的輸入資料型態不同，因此需要先對齊比較基準。

jhash2

`jhash2()` 的 length 單位是 `u32 words`，不是 bytes。

```
words = input_len / sizeof(u32);  
result = jhash2((const u32 *)buf, words, seed);
```

hsiphash

`hsiphash()` 使用 byte buffer 與 byte length。

```
result = hsiphash(buf, input_len, &hkey);
```

因此 `input_len` 必須是 4 bytes 的倍數，才能公平呼叫 `jhash2()`

若輸入長度未對齊，`jhash2` 會進行 truncation 忽略尾端的資料，導致處理的資料負載不同

實驗設計：SipHash output folding

`siphash()` 回傳 64-bit，但 OVS-like hash path 使用 32-bit。

因此本實驗將高 32-bit 與低 32-bit 以 XOR folding 合併。

```
u64 h = siphash(buf, input_len, &skkey);  
u32 folded = (u32)h ^ (u32)(h >> 32);
```

folding cost 需要納入 timing window。

原因是在實驗二，若 SipHash 實際放入 OVS Flow table 32-bit hash path，這個轉換成本也會發生。這不是為了改變 SipHash 本身，而是讓三種 hash 的輸出形式可以對齊到 32-bit hash value。

Hash loop 與 cycles/hash

module 內部用一個 timing window 包住整個 hash loop。

```
start = rdtsc_ordered();

for (i = 0; i < iterations; i++) {
    result = hash(buf, len, seed);
    acc += result;
}

end = rdtsc_ordered();
```

計算方式：

```
total_cycles = end - start
cycles_per_hash = total_cycles / iterations
```

例：10,000,000 次 hash 總共花 626,940,594 cycles
=> 平均每次 hash 約 62.69 cycles

Perf counter 與 multiplexing

除了 cycles/hash，使用 `perf stat` 量測硬體事件：

```
instructions, branch-misses, cache-misses
```

如教材〈運用 Perf 分析程式效能並改善〉中提及若同時量測太多 events，超出 CPU Performance Monitoring Unit (PMU) 的暫存器數量上限，將迫使 perf 啟動 Time Multiplexing 初期同時測六個事件，僅被量測 63% ~ 85% 的時間，後續數據用 perf scaling 估算，因此低於 100% counted 的量測。

Group	Events
core	<code>cycles,instructions</code>
branch	<code>branches,branch-misses</code>
cache	<code>cache-references,cache-misses</code>

採分組量測後，原始 log 顯示 measured events 為 `100.00% counted` 代表 multiplexing 問題已降低。

實驗結果：cycles/hash

Input lengths: 16, 32, 64, 128, 256 bytes

Iterations: 10,000,000 · Repeats: 10 · Event groups: core / branch / cache

Input	jhash2	hsiphash	siphash
16	16.712	28.554	45.376
32	26.869	38.341	64.480
64	61.426	57.203	101.947
128	119.872	95.860	174.763
256	249.810	171.048	324.874

觀察：SipHash 在所有 input length 下 cycles/hash 最高

jhash2 僅在小長度 (≤ 32 bytes) 具備優勢，當長度達 64 bytes 時，效能發生反轉。

實驗結果：instructions/hash

Input	jhash2	hsiphash	siphash
16	83.3841	148.3923	207.4183
32	118.3891	190.4117	279.4423
64	235.4635	274.4387	423.5020
128	422.5506	442.5090	711.6016
256	843.7541	778.6210	1287.8416

觀察：SipHash 的 instructions/hash 也最高，與 cycles/hash 趨勢一致。

這表示 SipHash 的額外成本主要來自較多計算指令與較重的 keyed mixing / finalization。
另外，在 256 bytes 時 `hsiphash` 的 instructions/hash 低於 `jhash2`。

Branch / Cache Counters

Metric	jhash2	hsiphash	siphash
branch-misses/hash	0.0019–0.0038	0.0020–0.0035	0.0021–0.0049
cache-misses/hash	0.0083–0.0112	0.0089–0.0108	0.0092–0.0116

範圍取自 input length = 16, 32, 64, 128, 256 bytes。

- `branch-misses/hash` 整體低於 0.005，表示此 benchmark 中幾乎沒有大量 branch prediction failure。
- `cache-misses/hash` 大多落在 0.008–0.012，且沒有隨 SipHash 的 cycles/hash 明顯增加。
- SipHash 的額外成本主要來自較多 instructions 與較重的 keyed hash computation，branch prediction 或 cache locality 並非主因，但詳細情況仍須進一步分析。

結論

SipHash 在所有 input length 下都有最高的 cycles/hash 與 instructions/hash。

這表示 SipHash 的額外成本主要反映在：

- higher instruction count
- heavier keyed mixing / finalization
- 64-bit to 32-bit folding conversion

目前主要成本差異較能由 instruction count 與 hash computation complexity 解釋。

實驗侷限：HalfSipHash 在較長 input 下 cycles/hash 低於 jhash2，這裡先視為本平台、本實作路徑下的觀察，不作為通用結論。

TODO

已完成：

- Experiment 1: kernel-side microbenchmark
- cycles/hash
- instructions/hash
- branch-misses/hash
- cache-misses/hash
- grouped perf measurement with 100.00% counted

後續實驗：

- 以 OVS `flow_hash()` 為對象
- 測試 OVS datapath throughput / CPU usage
- 分析 bucket distribution 與 collision behavior